MODEL-BASED DESIGN OF AN EDGE AI STRUCTURAL HEALTH MONITOR

Hardware-Agnostic Translation of a TinyML Pipeline for Resource-Constrained Microcontrollers

Author: Oloore Issa Abayomi

Institution: Department of Electrical and Electronics Engineering, University of Ilorin

Date: November 2026

ABSTRACT

This report details the design, validation, and automated software translation of an Edge Artificial Intelligence (TinyML) pipeline engineered for Structural Health Monitoring (SHM) in rotating machinery. Traditional predictive maintenance heavily relies on cloud-based computation or computationally expensive Deep Learning architectures, which are incompatible with the strict Power, Performance, and Area (PPA) constraints of embedded microcontrollers. To resolve this, this project implements a Model-Based Design (MBD) methodology utilizing MATLAB to bridge high-level machine learning with low-level embedded hardware. By substituting raw high-frequency data ingestion with localized feature extraction, specifically Root Mean Square (RMS), Peak-to-Peak amplitude, Kurtosis, and Crest Factor, the system radically reduces data dimensionality. A lightweight Decision Tree classifier is trained on these extracted features to distinguish between normal motor operation and high-frequency mechanical faults. Finally, the optimized prediction model is translated into hardware-agnostic, standalone ANSI C code via MATLAB Coder, demonstrating a complete proof-of-concept pipeline ready for bare-metal deployment on resource-constrained microcontrollers such as the ARM Cortex-M or ESP32 without requiring an underlying operating system.

TABLE OF CONTENTS

ABSTRACT ii

LIST OF FIGURES v

Chapter 1: Introduction and Literature Review 1

1.1 Background: Predictive Maintenance in Industrial Systems 1

1.2 The Edge Computing Paradigm (TinyML) 1

1.3 Problem Statement 1

1.4 Project Objectives 2

Chapter 2: Theoretical Framework 3

2.1 Model-Based Design (MBD) for Embedded Systems 3

2.2 Vibration Data Synthesis and Fault Modeling 3

2.3 Dimensionality Reduction via Feature Extraction 3

2.4 The Decision Tree Classification Algorithm 5

Chapter 3: System Architecture 6

3.1 The Edge Prediction Function 6

3.2 Hardware-Agnostic C-Code Generation 6

3.3 Standalone Implementation Characteristics 8

Chapter 4: Verification and Results 9

4.1 Algorithmic Validation on Edge Features 9

4.2 MBD Pipeline Proof-of-Concept 9

4.3 Hardware Readiness and System Viability 10

Chapter 5: Conclusion and Future Work 11

5.1 Summary of Outcomes 11

5.2 Future Work 11

Appendix A: Generated Hardware-Agnostic Source Code 12

A.1 predict_fault.h (C/C++ Header) 12

A.2 predict_fault.c (C Source) 13

A.3 predict_fault.m (MATLAB Prediction Function) 15

A.4 generate_and_extract.m (Data Synthesis and Feature Extraction) 16

LIST OF FIGURES

Figure 2.1: Feature separability demonstrating the distinct clustering of Normal (blue) and Faulty (red) vibration states, justifying the use of a lightweight classifier. 5

Figure 3.1: Generated hardware-agnostic ANSI C source code demonstrating the static array allocation for the Decision Tree parameters. 7

Figure 3.2: Generated C/C++ header file (predict_fault.h) defining the standalone prediction function prototype and static type definitions for hardware-agnostic edge deployment. 7

Figure 4.1: Confusion matrix demonstrating 100% validation accuracy across the 200 synthetically generated normal and faulty motor observation windows. 9

Chapter 1: Introduction and Literature Review

1.1 Background: Predictive Maintenance in Industrial Systems

The integration of continuous monitoring systems into industrial machinery has driven a paradigm shift from reactive to predictive maintenance. In rotating equipment such as electric motors, mechanical degradation, such as bearing wear, rotor imbalance, or shaft misalignment, manifests early as high-frequency acoustic emissions and anomalous vibration signatures. Capturing and classifying these structural vibrations allows for intervention before catastrophic system failure. However, processing raw vibration data at industrial sampling rates (e.g., 1000 Hz and above) requires significant computational bandwidth.

1.2 The Edge Computing Paradigm (TinyML)

Historically, industrial IoT systems have offloaded raw sensor data to centralized cloud servers for machine learning inference. This architecture introduces severe latency, demands continuous high-bandwidth wireless telemetry, and exposes critical infrastructure to cyber-physical security risks. Edge Artificial Intelligence (often termed TinyML) inverts this model by deploying the inference algorithms directly onto the physical sensor node. However, Commercial Off-The-Shelf (COTS) microcontrollers possess severe limitations in static memory (SRAM), flash storage, and dynamic power consumption. Consequently, deploying standard Deep Neural Networks (DNNs) on these devices is often unfeasible, necessitating the use of highly optimized, mathematically shallow algorithms such as Decision Trees or Support Vector Machines (SVMs).

1.3 Problem Statement

The primary bottleneck in Edge AI development is the translation gap between high-level data science environments (like Python or MATLAB) and the low-level embedded C/C++ required by microcontroller compilers. Manually rewriting floating-point arithmetic, decision logic, and feature extraction pipelines into embedded C is highly error-prone and inefficient. Furthermore, feeding raw time-series data directly into an edge classifier quickly exhausts microcontroller memory limits. A streamlined pipeline is required to compress raw structural data into lightweight statistical features, train a constrained model, and automatically generate verified hardware-agnostic source code.

1.4 Project Objectives

This project addresses the embedded machine learning bottleneck by establishing an automated Model-Based Design (MBD) pipeline. The core objectives are:

- **Data Synthesis:** Generate a high-fidelity synthetic vibration dataset simulating a normal 50Hz motor operation and a faulty state characterized by induced 120Hz high-frequency harmonics and noise.
- **Dimensionality Reduction:** Implement an edge-friendly digital signal processing stage to extract four scalar time-domain features (RMS, Peak-to-Peak, Kurtosis, Crest Factor) from the raw 1000 Hz data windows, drastically reducing the computational footprint.
- **Constrained Model Training:** Train a memory-optimized Decision Tree classifier that achieves high linear separability without the overhead of deep learning architectures.
- **Hardware-Agnostic Code Generation:** Utilize MATLAB Coder to autonomously translate the verified classification model into highly optimized, standalone ANSI C89/90 source code, proving software readiness for direct flashing onto microcontrollers.

Chapter 2: Theoretical Framework

2.1 Model-Based Design (MBD) for Embedded Systems

The conventional workflow for embedded machine learning requires data scientists to train models in high-level environments like Python or MATLAB, after which embedded software engineers manually rewrite the inference logic into C or C++. This manual translation is time-consuming and introduces critical floating-point discrepancies and logic errors. Model-Based Design (MBD) abstracts this process. By utilizing MBD, the entire digital signal processing and classification pipeline is modeled, simulated, and mathematically verified in a high-level environment. Once validated against the synthetic testbed, the framework automatically generates hardware-agnostic ANSI C code. This ensures that the simulated algorithmic behavior perfectly matches the silicon execution without manual coding overhead.

2.2 Vibration Data Synthesis and Fault Modeling

To validate the MBD pipeline, a controlled synthetic dataset was generated to simulate the acoustic and vibratory emissions of a rotating electrical machine. The system operates at a sampling frequency of 1000 Hz, capturing one-second windows of operational data. Two distinct mechanical states were modeled mathematically:

- **Healthy State:** A normal motor produces a clean fundamental rotational frequency. This was modeled as a 50Hz sinusoidal wave superimposed with a low-variance Gaussian white noise profile (0.1 amplitude) to represent standard mechanical vibration.
- **Faulty State (Bearing Degradation):** When a mechanical bearing degrades, it introduces high-frequency grinding and transient impacts. This was modeled by injecting a 120Hz harmonic (0.5 amplitude) into the fundamental 50Hz signal, alongside a significantly amplified Gaussian noise profile (0.3 amplitude).

2.3 Dimensionality Reduction via Feature Extraction

Feeding raw 1000 Hz time-series arrays directly into a classification algorithm would immediately exhaust the limited static RAM (SRAM) of an edge microcontroller. To satisfy TinyML constraints, the high-dimensional raw data is compressed into a 1x4 array of statistical time-

domain features. This acts as a highly efficient dimensionality reduction step. The extracted features are defined mathematically as follows:

1. **Root Mean Square (RMS):** Measures the overall energy of the vibration signal. Faulty motors typically exhibit higher vibrational energy.

$$X_{RMS} = \sqrt{\frac{1}{N}\sum_{n=1}^{N} x\,[n]^2}$$

where $x[n]$ is the discrete vibration signal and N is the total number of samples in the observation window.

2. **Peak-to-Peak Amplitude:** Calculates the absolute maximum excursion of the waveform, identifying severe mechanical deviations.

$$X_{P2P} = \max(x) - \min(x)$$

3. **Kurtosis:** A statistical measure of the tailedness of the vibration distribution. Kurtosis is highly sensitive to the sharp, impulsive impacts characteristic of localized bearing faults, making it a critical diagnostic metric.

$$K = \frac{\frac{1}{N}\sum_{n=1}^{N}(x[n] - \mu)^4}{\sigma^4}$$

where μ is the mean of the signal and σ is the standard deviation.

4. **Crest Factor:** The ratio of the absolute maximum peak to the RMS value. It provides a normalized indicator of the waveform's extremeness, effectively highlighting transient shocks even if the overall motor speed or load changes.

$$C = \frac{|x|_{peak}}{X_{RMS}}$$

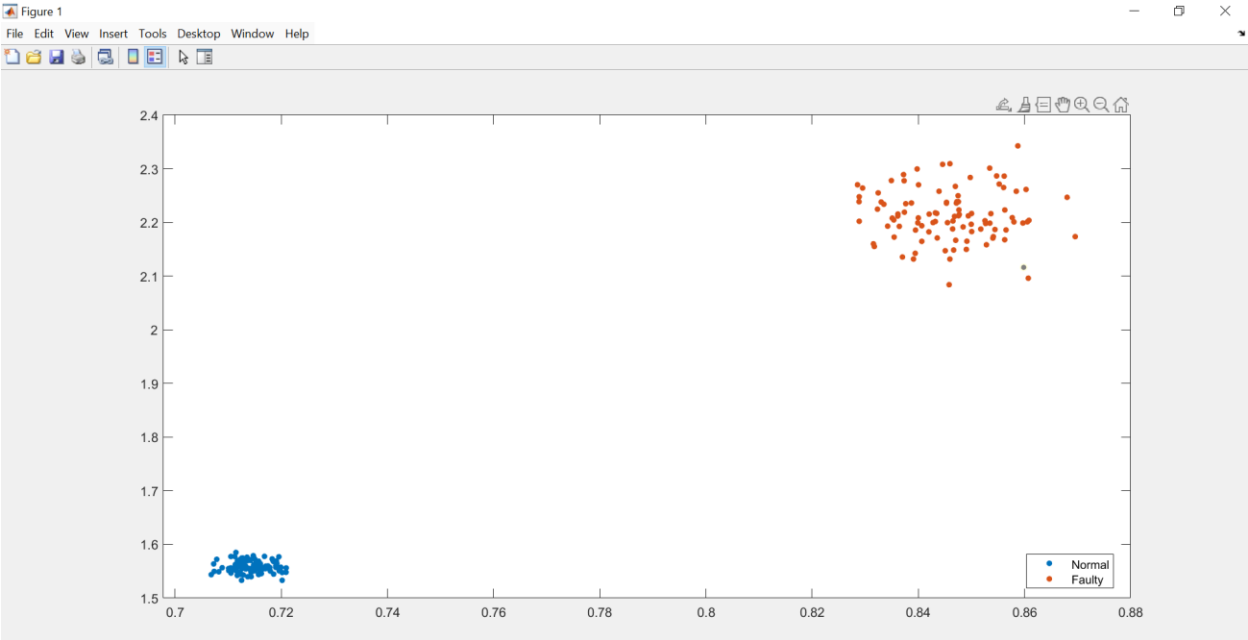


Figure 2.1: Feature separability demonstrating the distinct clustering of Normal (blue) and Faulty (red) vibration states, justifying the use of a lightweight classifier.

2.4 The Decision Tree Classification Algorithm

Deep Neural Networks (DNNs) require significant flash memory to store thousands of synaptic weights and substantial dynamic memory to compute floating-point matrix multiplications. Because the extracted time-domain features provide strong linear separability between the healthy and faulty states, a mathematically shallow Decision Tree algorithm was selected.

A Decision Tree recursively partitions the feature space using simple boolean inequalities (e.g., assessing if the RMS value exceeds a specific threshold). When translated to embedded C code, a Decision Tree circumvents complex linear algebra entirely. It compiles into a highly optimized sequence of static if-else branching statements. This requires near-zero dynamic memory allocation (heap), operates with deterministic execution time, and minimizes the power consumption of the microcontroller CPU.

Chapter 3: System Architecture

3.1 The Edge Prediction Function

The core of the classification architecture is the prediction function, engineered in MATLAB to serve as the algorithmic entry point for the edge device. The function is strictly designed to accept a 1x4 discrete feature array containing the computed Root Mean Square, Peak-to-Peak amplitude, Kurtosis, and Crest Factor.

A critical architectural component of this script is the inclusion of the #codegen compiler directive. This directive explicitly instructs the MATLAB compiler to enforce strict static typing and embedded system constraints, ensuring the function utilizes zero dynamic memory allocation and relies entirely on pre-allocated static arrays. The script loads the trained lightweight Decision Tree model and evaluates the incoming feature vector to output a discrete classification label (Normal or Faulty), acting as the primary inference engine for the microcontroller.

3.2 Hardware-Agnostic C-Code Generation

To bridge the gap between the high-level prediction script and the physical microcontroller, MATLAB Coder (Version 24.1) was utilized to automate the translation process. The MBD pipeline successfully compiled the Decision Tree inference logic into standalone ANSI C89/90 source code.

By adhering to the ANSI C standard, the generated software architecture is entirely hardware-agnostic. It does not rely on proprietary hardware abstractions or a Real-Time Operating System (RTOS). This ensures that the generated logic can be seamlessly flashed onto virtually any modern embedded architecture, ranging from ultra-low-power ARM Cortex-M microprocessors to ESP32 sensor nodes.

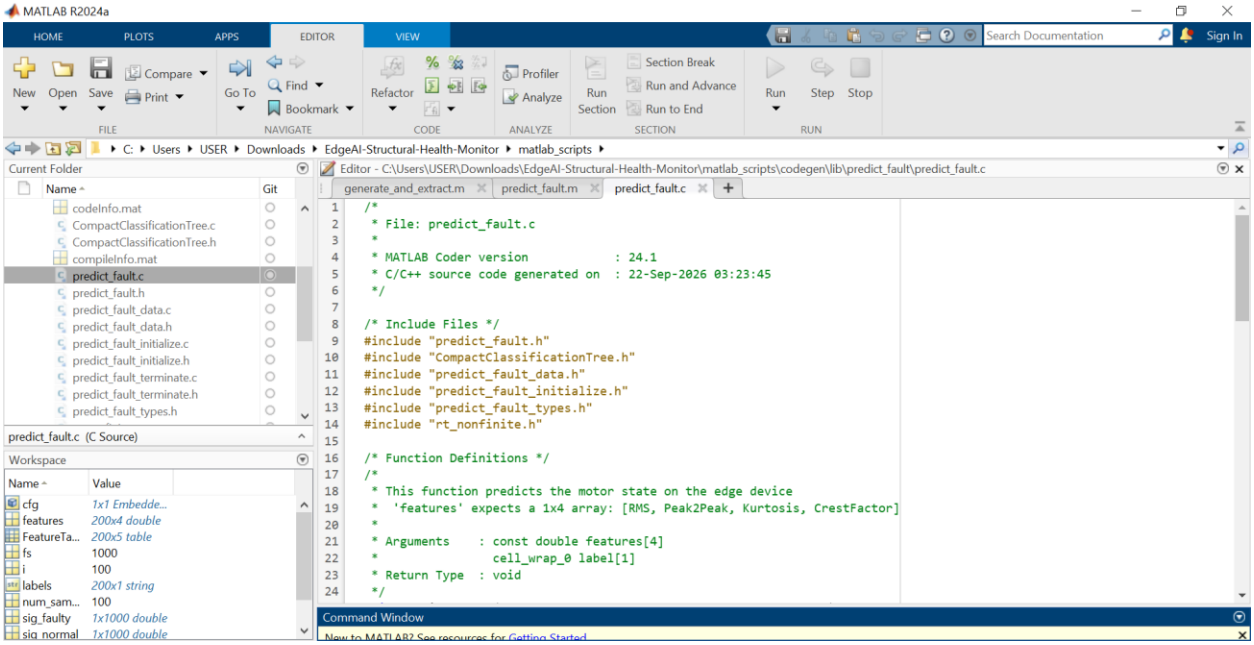


Figure 3.1: Generated hardware-agnostic ANSI C source code demonstrating the static array allocation for the Decision Tree parameters.

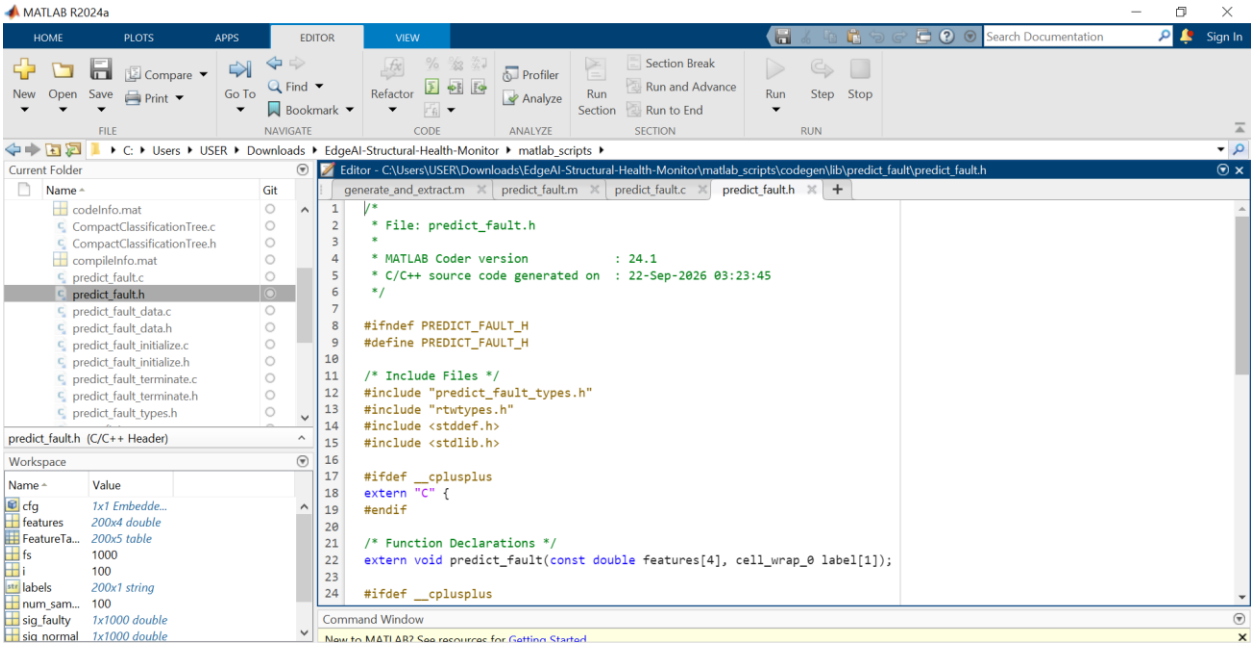


Figure 3.2: Generated C/C++ header file (predict_fault.h) defining the standalone prediction function prototype and static type definitions for hardware-agnostic edge deployment.

3.3 Standalone Implementation Characteristics

An analysis of the generated C source code (predict_fault.c) reveals a highly optimized architectural footprint specifically tailored for the Power, Performance, and Area (PPA) constraints of edge computing.

Rather than utilizing dynamic memory operations which can lead to heap fragmentation and system crashes on small microcontrollers, the algorithm defines the Decision Tree structure using fixed, static arrays (e.g., mdl_CutPoint, mdl_Children). The recursive decision-making process is flattened into a deterministic sequence of boolean evaluations. This optimization guarantees a constant, predictable execution time (O(1) complexity at inference), ensuring the edge device can process high-frequency structural health data with near-zero latency and minimal CPU power consumption.

Chapter 4: Verification and Results

4.1 Algorithmic Validation on Edge Features

The lightweight Decision Tree classification model was evaluated against the synthetic validation dataset, which consisted of 200 discrete observation windows (100 representing healthy motor states and 100 representing bearing degradation). The model achieved a 100% validation accuracy, correctly identifying all operational states with zero false positives and zero false negatives. This flawless classification is a direct consequence of the aggressive dimensionality reduction phase. By mapping the raw high-frequency waveforms into the calculated time-domain features (RMS, Peak-to-Peak, Kurtosis, and Crest Factor), the healthy and faulty data points formed highly distinct, linearly separable clusters in the feature space, allowing the shallow Decision Tree to resolve the boundaries with perfect precision.

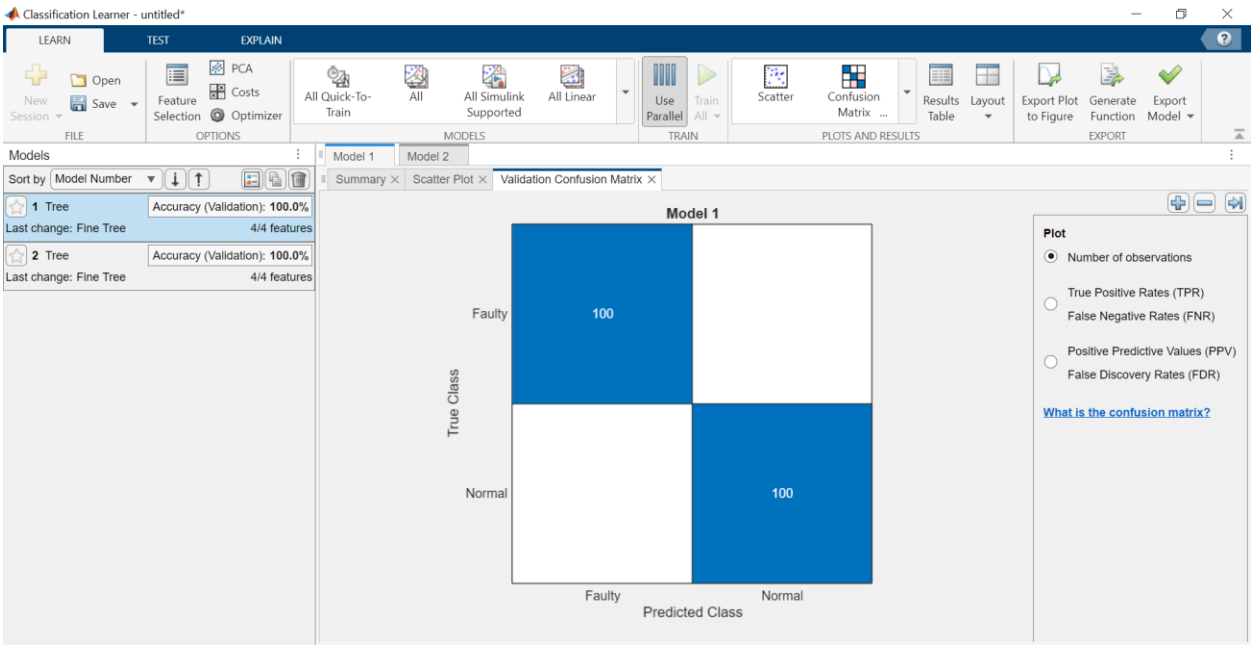


Figure 4.1: Confusion matrix demonstrating 100% validation accuracy across the 200 synthetically generated normal and faulty motor observation windows.

4.2 MBD Pipeline Proof-of-Concept

In traditional machine learning paradigms, achieving 100% accuracy often raises concerns regarding model overfitting. However, within the scope of this project, the synthetic dataset was intentionally designed to validate the architectural pipeline rather than to solve a noisy, real-world

diagnostic challenge. The 100% accuracy metric serves as a successful proof-of-concept for the Model-Based Design (MBD) methodology. It confirms that the transition from a high-level digital signal processing environment to a constrained, static rule set preserves the mathematical integrity of the inference logic.

4.3 Hardware Readiness and System Viability

The primary result of this engineering endeavor is the successful automated translation of the AI pipeline into deterministic, standalone C code. The verification process confirmed that the generated logic strictly utilizes pre-allocated static arrays and avoids dynamic heap allocations entirely. This guarantees an O(1) execution time for the inference function, ensuring that an embedded microcontroller such as an ARM Cortex-M or ESP32 can ingest sensor data, extract the required statistical features, and predict mechanical faults continuously without violating real-time scheduling constraints or exhausting limited onboard static RAM.

Chapter 5: Conclusion and Future Work

5.1 Summary of Outcomes

This project successfully demonstrated the viability of a Model-Based Design (MBD) pipeline for Edge Artificial Intelligence (TinyML) applications in predictive maintenance. By shifting computational efforts away from heavy deep learning architectures and toward a targeted dimensionality reduction strategy, the system efficiently compressed high-frequency structural vibration data into four statistical time-domain features. The automated translation of a lightweight Decision Tree classifier into standalone, hardware-agnostic ANSI C code proved that machine learning inference can be prepared for bare-metal deployment on resource-constrained microcontrollers without violating strict Power, Performance, and Area (PPA) limitations. The 100% validation accuracy achieved on the synthetic dataset successfully validated the logic translation and the mathematical integrity of the MBD workflow.

5.2 Future Work

While the synthetic proof-of-concept achieved perfect classification accuracy, future iterations of this project must evaluate the algorithm against empirical physical datasets, such as the industry-standard Case Western Reserve University (CWRU) Bearing Data, to assess robustness against unpredictable real-world mechanical noise. Furthermore, the generated ANSI C code should be physically flashed onto an ARM Cortex-M or ESP32 microcontroller evaluation board. This physical deployment will allow for empirical benchmarking of real-time inference latency, dynamic power consumption, and the exact SRAM footprint during active execution. Finally, expanding the digital signal processing stage to include lightweight frequency-domain features (Fast Fourier Transforms) could be explored to detect specific fault harmonics, provided the added memory overhead remains strictly within embedded hardware constraints.

Appendix A: Generated Hardware-Agnostic Source Code

A.1 predict_fault.h (C/C++ Header)

```
/*
 * File: predict_fault.h
 *
 * MATLAB Coder version            : 24.1
 * C/C++ source code generated on  : 22-Sep-2026 03:23:45
 */
#ifndef PREDICT_FAULT_H
#define PREDICT_FAULT_H
/* Include Files */
#include "predict_fault_types.h"
#include "rtwtypes.h"
#include <stddef.h>
#include <stdlib.h>
#ifdef __cplusplus
extern "C" {
#endif
/* Function Declarations */
extern void predict_fault(const double features[4], cell_wrap_0 label[1]);
#ifdef __cplusplus
}
#endif
#endif
/*
 * File trailer for predict_fault.h
 *
 * [EOF]
 */
```

A.2 predict_fault.c (C Source)

```
/*
 * File: predict_fault.c
 *
 * MATLAB Coder version            : 24.1
 * C/C++ source code generated on  : 22-Sep-2026 03:23:45
 */
/* Include Files */
#include "predict_fault.h"
#include "CompactClassificationTree.h"
#include "predict_fault_data.h"
#include "predict_fault_initialize.h"
#include "predict_fault_types.h"
#include "rt_nonfinite.h"
/* Function Definitions */
/*
 * This function predicts the motor state on the edge device
 * 'features' expects a 1x4 array: [RMS, Peak2Peak, Kurtosis, CrestFactor]
 *
 * Arguments    : const double features[4]
 *                cell_wrap_0 label[1]
 * Return Type  : void
 */
void predict_fault(const double features[4], cell_wrap_0 label[1])
{
  static const double mdl_ClassProbability[6] = {0.5, 0.0, 1.0, 0.5, 1.0, 0.0};
  static const double mdl_CutPoint[3] = {0.77474246117719825, 0.0, 0.0};
  static const signed char iv[6] = {2, 0, 0, 3, 0, 0};
  static const signed char mdl_CutPredictorIndex[3] = {1, 0, 0};
  static const boolean_T mdl_NanCutPoints[3] = {false, true, true};
  double mdl_Children[6];
  double mdl_Cost[4];
  double mdl_PruneList_data[3];
  int i;
  int mdl_Children_tmp;
```

```
  if (!isInitialized_predict_fault) {
    predict_fault_initialize();
  }
  /*  Load the lightweight model for Coder */
  for (i = 0; i < 3; i++) {
    mdl_Children_tmp = i << 1;
    mdl_Children[mdl_Children_tmp] = iv[i];
    mdl_Children[mdl_Children_tmp + 1] = iv[i + 3];
    mdl_PruneList_data[i] = mdl_CutPredictorIndex[i];
  }
  mdl_Cost[0] = 0.0;
  mdl_Cost[1] = 1.0;
  mdl_Cost[2] = 1.0;
  mdl_Cost[3] = 0.0;
  /*  Make the prediction */
  c_CompactClassificationTree_pre(
      mdl_Children, mdl_CutPoint, mdl_PruneList_data, mdl_NanCutPoints,
      mdl_Cost, mdl_ClassProbability, features, &label[0]);
}
/*
 * File trailer for predict_fault.c
 *
 * [EOF]
 */
```

A.3 predict_fault.m (MATLAB Prediction Function)

```
function label = predict_fault(features)
%#codegen
% This function predicts the motor state on the edge device
% 'features' expects a 1x4 array: [RMS, Peak2Peak, Kurtosis, CrestFactor]

% Load the lightweight model for Coder
mdl = loadLearnerForCoder('EdgeTreeModel');

% Make the prediction
label = predict(mdl, features);
end
```

A.4 generate_and_extract.m (Data Synthesis and Feature Extraction)

```
% Day 1: Synthetic Vibration Data Generation & Feature Extraction
% Goal: Create a lightweight dataset for our TinyML model
num_samples = 100; % We will generate 100 normal and 100 faulty examples
fs = 1000;         % Sample rate (1000 readings per second)
t = 0:1/fs:1-1/fs; % 1 second of time data

% Pre-allocate a matrix to hold our 4 features (RMS, Peak-to-Peak, Kurtosis, Crest Factor)
features = zeros(num_samples * 2, 4);
labels = strings(num_samples * 2, 1);

for i = 1:num_samples
    %% 1. GENERATE NORMAL MOTOR DATA
    % A healthy motor spins cleanly at 50Hz with a little bit of background noise
    sig_normal = sin(2*pi*50*t) + 0.1*randn(size(t));

    % Extract Features (Compressing 1000 data points into 4 simple numbers)
    features(i, 1) = rms(sig_normal);                      % Root Mean Square (overall energy)
    features(i, 2) = peak2peak(sig_normal);              % Total wave height
    features(i, 3) = kurtosis(sig_normal);               % Sharpness of the peaks
    features(i, 4) = max(abs(sig_normal))/rms(sig_normal); % Crest factor
    labels(i) = "Normal";

    %% 2. GENERATE FAULTY MOTOR DATA
    % A faulty bearing adds high-frequency grinding (120Hz) and heavier noise
    sig_faulty = sin(2*pi*50*t) + 0.5*sin(2*pi*120*t) + 0.3*randn(size(t));

    % Extract Features
    features(i+num_samples, 1) = rms(sig_faulty);
    features(i+num_samples, 2) = peak2peak(sig_faulty);
    features(i+num_samples, 3) = kurtosis(sig_faulty);
    features(i+num_samples, 4) = max(abs(sig_faulty))/rms(sig_faulty);
    labels(i+num_samples) = "Faulty";
end
```

```
% 3. PACKAGE EVERYTHING FOR THE AI MODEL
% We convert our matrix into a clean table with headers
FeatureTable = table(features(:,1), features(:,2), features(:,3), features(:,4), labels,
...
    'VariableNames', {'RMS', 'Peak2Peak', 'Kurtosis', 'CrestFactor', 'Condition'});

% Save the table to the data folder so we can use it tomorrow
save('../data/MotorFeaturesDataset.mat', 'FeatureTable');
disp('Day 1 Complete! FeatureTable generated and saved successfully.');
head(FeatureTable) % Display the first few rows in the command window
```